4. Modelowanie wymagań

4.1. Przegląd rozdziału

4.1.1. Czego dotyczy ten rozdział?

Modelowanie wymagań jest jedną z trudniejszych prac do wykonania w czasie budowy systemu informatycznego. Jednocześnie błędy popełniane w tej fazie prac skutkują największymi kłopotami i kosztami w kolejnych fazach wytwarzania oprogramowania. W tym rozdziale skoncentrujemy się na przedstawieniu najważniejszych elementów związanych z modelowaniem wymagań. Nie będziemy jednak zajmować się opisem wymagań za pomocą diagramów przypadków użycia. Opis diagramów przypadków użycia oraz dyskusję na temat dobrych i złych technik z zastosowaniem tych diagramów znajdzie Czytelnik w rozdziale 8.

4.1.2. Co powinieneś wiedzieć, aby zrozumieć?

Ten rozdział w zasadzie nie wymaga od Czytelnika żadnej wiedzy bazowej. Wystarczą nam odwołania do ogólnej wiedzy i doświadczenia Czytelnika.

4.1.3. Jakie najważniejsze problemy rozwiążemy w tym rozdziale?

Jednym z podstawowych problemów, z jakimi muszą poradzić sobie twórcy systemów informatycznych, jest prawidłowe rozpoznanie i udokumentowanie wymagań (oczekiwań użytkownika) wobec przyszłego systemu.

W tym rozdziale staramy się odpowiedzieć na pytanie, czym są wymagania oraz w jaki sposób można je sklasyfikować. Najważniejsze zaś pytaniem, na które staramy się udzielić odpowiedzi jest pytanie: w jaki sposób dokumentować wymagania związane z funkcjonalnością systemu?

4.1.4. Czego dowiesz się po jego przeczytaniu?

Po przeczytaniu tego rozdziału Czytelnik dowie się czym są wymagania dla systemu informatycznego, w jaki sposób można podzielić wymagania oraz jak można dokumentować wymagania w dokumencie wymagań.

4.1.5. Dlaczego warto przeczytać ten rozdział?

Błędy popełniane w fazie analizy i definiowania wymagań są niezwykle trudne do naprawienia. Dlatego też warto jest poświęcić czas i uwagę, aby prawidłowo zidentyfikować wymagania oraz je dobrze opisać.

Rozdział ten powinien uzmysłowić Czytelnikowi czym są wymagania i jak w sposób jasny i użyteczny dla dalszego procesu wytwarzania oprogramowania je dokumentować.

4.2. Czym są wymagania dla systemu informatycznego?

Określanie wymagań klienta wobec przyszłego systemu jest zadaniem o kluczowym znaczeniu dla powodzenia projektu. Jest też zadaniem niezwykle trudnym, ze względu na ciągłą zmienność wymagań oraz trudności w uświadomieniu i zwerbalizowaniu przez klienta jego oczekiwań w stosunku do projektowanego systemu.

Faza określania wymagań, w której dokonywana jest identyfikacja wymagań oraz ich dokumentacja w sposób możliwie precyzyjny powinna być przeprowadzana przy znacznym zaangażowaniu przedstawicieli klienta. Najlepszym podejściem jest tutaj model iteracyjny.

Celem fazy określenia wymagań jest ustalenie wymagań klienta wobec tworzonego systemu i zapisanie ich w postaci zrozumiałego dla obydwu stron (klienta i wykonawcy), spójnego dokumentu.

Podczas identyfikowania i definiowania wymagań dokonywana jest zamiana celów klienta określonych w fazie strategicznej na konkretne wymagania zapewniające osiągnięcie tych celów.

Z powodu trudności w pozyskiwaniu wymagań, trudności w ustaleniu, jakie wymagania zapewnią osiągniecie jego celów i innych czynników, o których piszemy dalej, faza określania wymagań nie jest prostym zbieraniem wymagań, lecz procesem, w którym klient wspólnie z przedstawicielem producenta konstruuje zbiór wymagań (w zgodzie z postawionymi celami).

W przypadku systemu na zamówienie analitycy mają bezpośredni kontakt z przedstawicielami klienta. Faza określania dużego zaangażowania ze strony klienta, przyszłych użytkowników systemu i ekspertów w danej dziedzinie.

Jak już zostało wspomniane, określanie wymagań jest etapem o bardzo dużym znaczeniu dla powodzenia projektu, a jednocześnie etapem niezwykle trudnym do poprawnej realizacji. Trudności w określaniu i zbieraniu wymagań wynikają z wielu czynników.

Pierwsza trudność polega na tym, że klient z reguły nie wie dokładnie, w jaki sposób osiągnąć założone cele strategiczne, czyli jakie powinny być wymagania odnośnie funkcjonowania systemu. Dochodzi do tego jeszcze niejednoznaczność rozwiązań polegająca na tym, że cele klienta mogą być osiągnięte na wiele sposobów.

Po drugie, aby osiągnąć sukces, konieczne jest wzajemne zrozumienie między klientem a wykonawcą. Aby wykonawca mógł zaprojektować i wykonać system spełniający prawidłowo oczekiwania klienta, musi je dobrze poznać i zrozumieć. Dlatego tak ważny jest dialog między użytkownikiem a zespołem projektowym.

Klient musi w sposób precyzyjny, spójny i zrozumiały dla obydwu stron wyrazić swoje oczekiwania wobec budowanego systemu. Niestety, zwerbalizowanie wymagań przez klienta, nawet gdy wie on, czego oczekuje od systemu, nie zawsze jest łatwe.

Dodatkowo duże systemy są wykorzystywane przez wielu użytkowników. Ich cele są często sprzeczne. Różni użytkownicy mogą posługiwać się inną terminologią, mówiąc o tych samych problemach.

Zleceniodawcy i użytkownicy to często inne osoby. Głos zleceniodawców może być w tej fazie decydujący, chociaż nie zawsze potrafią oni właściwie przewidzieć potrzeby przyszłych użytkowników.

Rola projektanta sytemu polega na poszukiwaniu i uzgadnianiu licznych kompromisów. Kompromisy te dotyczą wielu obszarów, począwszy od kompromisów w definiowaniu wymagań (wynikających na

przykład z różnych potrzeb różnych użytkowników) poprzez kompromisy technologiczne aż do kompromisów wynikających z ograniczonych zasobów finansowych lub ograniczeń czasowych.

Podczas określania wymagań mamy do czynienia z jednej strony z trudnościami w samej identyfikacji i wyborze wymagań, z drugiej zaś z ich udokumentowaniem w taki sposób, aby mogły one posłużyć do zaprojektowania i wykonania oczekiwanego systemu.

4.3. Rodzaje wymagań

Wymagania wobec systemów informatycznych możemy podzielić na dwie grupy:

- wymagania funkcjonalne,
- wymagania niefunkcjonalne.

4.3.1. Wymagania funkcjonalne

Wymagania funkcjonalne opisują funkcje (czynności, operacje) wykonywane przez system. Funkcje te mogą być również wykonywane przy użyciu systemów zewnętrznych.

Określenie wymagań funkcjonalnych obejmuje następujące kwestie:

Określenie wszystkich rodzajów użytkowników (aktorów), którzy będą korzystać z systemu.

Określenie wszystkich rodzajów użytkowników, którzy są niezbędni do działania systemu (obsługa, wprowadzanie danych, administracja).

Dla każdego rodzaju użytkownika (aktora) określenie funkcji systemu oraz sposobów korzystania z planowanego systemu.

Określenie systemów zewnętrznych (obcych baz danych, sieci, Internetu), które będą wykorzystywane podczas działania systemu.

Ustalenie struktur organizacyjnych, przepisów prawnych, statutów, zarządzeń, instrukcji itd., które pośrednio lub bezpośrednio określają funkcje wykonywane przez planowany system.

Wszystkie wymagania funkcjonalne muszą być zebrane i zapisane w dokumencie wymagań funkcjonalnych. Dokument ten powinien być tak skonstruowany, aby zapewnić wymagania dotyczące jakości opisu wymagań przedstawione w punkcie 3.3. Podczas specyfikacji wymagań stosuje się różne techniki. Kilka z nich zostało wymienionych poniżej. Zazwyczaj w specyfikacji wymagań występuje kilka różnych technik opisu.

Język naturalny

Wymagania funkcjonalne na wstępnym etapie prac nad systemem są zazwyczaj dokumentowane za pomocą języka naturalnego. Jest to najbardziej naturalna forma zapisu oczekiwań użytkownika w stosunku do projektowanego systemu, a jej zrozumienie nie wymaga od użytkownika żadnej dodatkowej wiedzy czy umiejętności. Niestety, opis przy użyciu języka naturalnego może być

niejednoznaczny i powodować różne rozumienie tego samego tekstu. Utrudnia to wykrycie powiązanych ze sobą wymagań i powoduje trudności w wykryciu sprzeczności.

Język naturalny strukturalny

W celu wyeliminowania wieloznaczności języka naturalnego wprowadza się do języka naturalnego ograniczenia słownictwa i składni oraz narzuca pewną strukturę opisu w postaci specyfikowania tematów i zagadnień w punktach i podpunktach. Taka forma opisu pozwala korzystać z zalet języka naturalnego, jednocześnie wymuszając większą jednoznaczność i spójność opisu. Nie eliminuje jednak całkowicie wad opisu języka naturalnego. Wymaga też od osób stosujących tę formę znajomości ograniczeń i umiejętności konstruowania specyfikacji przy tych ograniczeniach.

Tablice i formularze

Metoda ta polega na specyfikowaniu wymagań w postaci tablic (zwykle dwuwymiarowych), kojarzących różne aspekty (np. tablica ustalająca zależność pomiędzy typem użytkownika a rodzajem usługi). Tablice i formularze są bardzo pomocne, gdyż pozwalają strukturalizować informację i narzucają pewien reżim w zbieraniu i dokumentowaniu wymagań.

Diagramy blokowe

Diagramy blokowe są graficzną formą pokazująca najczęściej cykl przetwarzania procesów. Diagramy stanowią uzupełniającą formę specyfikacji wymagań. Na ogół w diagramach blokowych nie ma narzuconej z góry semantyki symboli graficznych, należy więc zwrócić uwagę na odpowiednią legendę do tych diagramów i na stosowanie stałej, z góry ustalonej konwencji. Diagramy odgrywają bardzo ważną rolę w komunikacji specyfikacji między klientem a wykonawcą, a dobrze opracowane są nieocenioną pomocą.

Diagramy kontekstowe

Podobnie jak poprzednie, są graficzną formą specyfikacji wymagań, z tym że diagramy kontekstowe ukazują system w postaci jednego bloku oraz jego powiązania z otoczeniem, wejściem i wyjściem.

Diagramy przypadków użycia

Diagramy przypadków użycia przedstawiają w poglądowy sposób aktorów i funkcje systemu oraz powiązania między nimi. Są wygodną formą wspomagającą komunikację między uczestnikami procesu specyfikacji wymagań. Należy podkreślić, że diagramy mogą stanowić bardzo ważny element specyfikacji wymagań ułatwiający ich zrozumienie i przyspieszający studiowanie tej specyfikacji pod warunkiem, że są zrozumiałe dla wszystkich uczestników. Dlatego przy ich wykonywaniu istotne jest stosowanie jednej, zrozumiałej i powszechnie znanej konwencji. Bardzo wygodną i powszechnie stosowaną konwencją w tego typu opisach jest język UML. Należy też podkreślić, że najczęściej same diagramy, choć ułatwiają pracę ze specyfikacją wymagań, to jednak nie niosą ze sobą pełnej informacji o nich. Dlatego diagramy, np. diagramy przypadków użycia, muszą być uzupełniane o opisy tekstowe w postaci scenariuszy przypadków użycia.

Formalizm matematyczny

Formalizm matematyczny stosuje się rzadko i dla specyficznych celów. Może być przydatny przy opisie zasad wyliczania pewnych parametrów w systemie, algorytmów itp.

Formularz wymagań funkcjonalnych

Wymagania funkcjonalne dobrze jest zbierać w postaci formularza wymagań funkcjonalnych. Formularz taki zbudowany jest w postaci prostej tabeli ze z góry zdefiniowanymi polami. Informacje w tych polach wpisywane są w języku naturalnym lub języku naturalnym strukturalnym.

Zastosowanie formularza wymagań funkcjonalnych pozwala na:

łatwe stwierdzenie kompletności opisu,

jednoznaczną interpretację.

4.3.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne stanowią drugą grupę wymagań wobec projektowanego systemu. Opisują one ograniczenia, przy których system ma realizować swoje funkcje.

Do przykładowych grup wymagań funkcjonalnych należą:

Wymagania dotyczące produktu

Są to ograniczenia narzucane na oprogramowanie. Ograniczenia te mogą dotyczyć sposobu obsługi (np. komunikacja użytkownika z systemem wyłącznie za pomocą klawiatury), używanych protokołów (np. oprogramowanie musi wykorzystywać protokół http), wydajności (np. start systemu po co najwyżej 5 sekundach) itp.

Wymagania dotyczące procesu

Są to ograniczenia związane z rzeczywistymi procesami modelowanymi i wspieranymi w systemie informatycznym (np. proces realizacji harmonogramowania zleceń musi być zgodny ze standardem opisanym w dokumencie XXXA/96).

Wymagania zewnętrzne

Są to wymagania związane z systemami i środowiskiem zewnętrznym w stosunku do projektowanego systemu (np. system harmonogramowania musi współpracować z bazą danych systemu komputerowego działu marketingu opisaną w dokumencie YYYB/95. Niedopuszczalne są jakiekolwiek zmiany w strukturze tej bazy.).

Weryfikowanie wymagań niefunkcjonalnych

Wymagania niefunkcjonalne powinny być możliwe do zweryfikowania. Oznacza to, że powinna istnieć możliwość sprawdzenia (np. przez pomiar), czy system je rzeczywiście spełnia. Na przykład wymagań

„system ma być łatwy w obsłudze", „system ma być niezawodny", „system ma być dostatecznie szybki" itd. nie można zweryfikować.

Brak zdefiniowania kryteriów oceny wymagań niefunkcjonalnych jest bardzo często popełnianym błędem. Należy więc zwracać uwagę, aby dla każdego wymagania niefunkcjonalnego określić kryterium weryfikowalności spełnienia tego wymagania. Najlepiej, jeśli jest to kryterium ławo mierzalne, jak na przykład czas reakcji systemu lub liczba obsługiwanych jednocześnie transakcji.

Poniżej przedstawiono przykładowe miary dla kilku cech związanych z wymaganiami niefunkcjonalnymi.

Cecha	Weryfikowalne miary
Wydajność	Liczba transakcji obsłużonych w ciągu sekundy Czas odpowiedzi
Rozmiar	Liczba rekordów w bazie danych Wymagana pamięć dyskowa
Łatwość użytkowania	Czas niezbędny dla przeszkolenia użytkowników Rozmiar dokumentacji
Niezawodność	Prawdopodobieństwo błędu podczas realizacji transakcji Średni czas pomiędzy błędnymi wykonaniami Dostępność (procent czasu, w którym system jest dostępny) Czas restartu po awarii systemu Prawdopodobieństwo zniszczenia danych w przypadku awarii
Przenaszalność	Procent kodu zależnego od platformy docelowej Liczba platform docelowych Koszt przeniesienia na nową platformę

4.4. Metody pozyskiwania i dokumentowania wymagań

W celu zminimalizowania trudności związanych z pozyskaniem i udokumentowaniem wymagań stosuje się różnorodne metody ich pozyskiwania i dokumentowania. Należą do nich metody kart CRC, burzy mózgów, map pamięci (ang. *mind mapping*) oraz metody oparte na scenariuszach i przypadkach użycia.

W celu identyfikacji wymagań należy posłużyć się różnymi metodami, które mogą dostarczyć jak najwięcej informacji o potrzebach użytkowników związanych z projektowanym systemem i możliwości ich zaspokojenia.

Do podstawowej grupy metod rozpoznawania wymagań należą wywiady przeprowadzane z przyszłymi użytkownikami.

4.4.1. Wywiady i przeglądy

Wywiad może być przeprowadzany z pojedynczym użytkownikiem lub grupą użytkowników. Za każdym razem wywiad taki powinien być przygotowany (w postaci listy pytań) i podzielony na odrębne zagadnienia. Podział powinien pokrywać całość tematu.

Niezwykle istotny z punktu widzenia przydatności wywiadu jest prawidłowy (reprezentatywny) dobór jego uczestników. Wywiady powinny doprowadzić do zgody między nimi i akceptacji projektu.

Zarówno w wywiadach indywidualnych, jak i grupowych można stosować różne techniki, takie jak karty CRC, burze mózgów, techniki map myśli itp. Ich dobór zależy od tematu spotkania, etapu, na jakim przeprowadzamy wywiad, oraz upodobań i umiejętności uczestników.

Wywiady powinny być przeprowadzane przez osoby cechujące się zarówno znajomością technik modelowania wymagań, jak i umiejętnością dialogu z klientem.

4.4.2. Studia nad istniejącym oprogramowaniem

Dość często nowe oprogramowanie zastępuje stare lub też uzupełnia jego możliwości. Studia nad istniejącym oprogramowaniem (tak zwane oprogramowanie spadkowe) są bardzo cenne. Pomagają one ustalić wszystkie dobre i złe strony starego oprogramowania, przyzwyczajenia użytkowników i przebieg procesów biznesowych, które nowe oprogramowanie powinno wspierać.

Jeśli przed przystąpieniem do budowy nowego systemu istnieje możliwość przyjrzenia się staremu systemowi wspierającemu podobne procesylub też wykorzystywanemu przez użytkowników przyszłego systemu, to należy takie badania przeprowadzić.

4.4.3. Studia wymagań systemowych

Są stosowane w sytuacji, kiedy nowy system ma być częścią większego systemu. Zdarza się to dosyć często. Wówczas projektant musi uwzględnić w swoim projekcie pewne ograniczenia wynikające z konieczności połączenia systemów w jedną całość. O ile to możliwe, należy też dążyć do ujednolicenia filozofii obsługi systemów, zasad budowy interfejsów itp., aby użytkownik nie musiał dodatkowo uczyć się obsługi nowej części systemu.

4.4.4. Studia osiągalności

Metoda ta polega na określeniu realistycznych celów systemu i metod ich osiągnięcia. Nie wszystkie wymagania zidentyfikowane podczas prac w fazie określania wymagań są możliwe do realizacji (np. przy ograniczeniach budżetowych) lub też nie realizują w sposób optymalny celów systemu. Dlatego potrzebne są badania nad przydatnością zidentyfikowanych wymagań i określenie stopnia ich przydatności dla osiągnięcia celów strategicznych.

4.4.5. Prototypowanie

Metoda prototypowania polega na zbudowaniu prototypu systemu działającego w zmniejszonej skali, np. z uproszczonymi interfejsami, z ograniczeniami funkcjonalnymi, bez kontroli błędów itp. Zaletą prototypu jest możliwość szybkiego przedstawienia użytkownikom zamierzeń projektowych w widocznej i „namacalnej” formie. Prototypowanie jest dobrą formą weryfikacji prawidłowego rozumienia przez

projektantów wymagań stawianych przez użytkowników. Jest szczególnie często stosowane przy budowie interfejsów graficznych.

Wymagania powinny być zebrane w dokumencie wymagań. Dokument ten powinien być podstawą szczegółowego kontraktu między klientem a producentem oprogramowania. Powinien także pozwalać na weryfikację stwierdzającą, czy wykonany system rzeczywiście spełnia postawione wymagania.

Dokument wymagań powinien być zrozumiały dla obydwu stron, spójny i łatwy do zmodyfikowania.

Często producenci nie są zainteresowani precyzyjnym formułowaniem wymagań, które pozwoliłoby na rzeczywistą weryfikację powstałego systemu. Tego rodzaju polityka lub niedbałość może prowadzić do konfliktów.

Jakość dokumentu wymagań

Dokument wymagań stanowi punkt wyjścia do dalszych prac projektowych, a później implementacyjnych. Źle lub nieprecyzyjnie określone wymagania mogą prowadzić do tego, że zbudowany, gotowy już system nie będzie spełniał oczekiwań klienta. Od jakości dokumentu wymagań zależy w znacznej mierze jakość budowanego systemu i satysfakcja klienta. Dlatego też tworząc dokument wymagań, należy zwrócić uwagę na jego odpowiednią jakość.

O jakości dokumentu wymagań świadczą:

Styl

Dokument wymagań powinien być napisany językiem jasnym, jednoznacznym oraz zrozumiałym dla użytkowników i projektantów. Powinien też być zorganizowany w jasną strukturę, po której łatwo się poruszać.

Spójność

Dokument wymagań nie powinien zawierać sprzeczności w opisie wymagań. Należy dokonać przeglądu dokumentu pod tym kątem, aby uniknąć sytuacji polegającej na tym, że w różnych miejscach dokumentu są zapisane wzajemnie sprzeczne wymagania.

Modyfikowalność

Zmienność wymagań w czasie prac nad systemem jest w zasadzie nieunikniona. Dokument wymagań powinien być więc przygotowany na takie zmiany i umożliwiać ich łatwe wprowadzanie. Można to osiągnąć, formułując wszystkie wymagania w zrozumiały sposób w punktach, które mogą być wyizolowane z kontekstu i zastąpione przez inne.

Ewolucyjność

Możliwość dodawania nowych wymagań, możliwość ich modyfikacji.

Identyfikacja odpowiedzialności

Dokument wymagań powinien umożliwiać łatwe określenie, kto jest odpowiedzialny za całość dokumentu wymagań lub jego fragment, kto lub co jest przyczyną sformułowania danego wymagania. Jest to istotne w przypadku modyfikacji, np. zmiany zakresu lub kontekstu systemu. W dokumencie wymagań powinny zatem znajdować się zapisy o odpowiedzialności (autora, osoby kontrolującej, osoby zatwierdzającej itp.). Dzięki temu w wypadku problemów ze zrozumieniem lub koniecznością uzgadniania zmian łatwiejsze będzie dotarcie do osoby dysponującej odpowiednią wiedzą lub kompetencjami.

Jednym z najlepszych sposobów opisu wymagań dla systemu informatycznego jest metoda oparta na przypadkach użycia. W tym rozdziale nie będziemy omawiać samych diagramów przypadków użycia, gdyż ich szczegółową dyskusję Czytelnik może znaleźć w rozdziale dziewiątym naszej książki. Skoncentrujemy się za to na opisie wymagań w języku naturalnym.

Poniżej przedstawimy w skrócie kroki, które powinien wykonać analityk w czasie wstępnej identyfikacji wymagań.

Krok pierwszy

Pierwszym krokiem w modelowaniu wymagań powinno być zidentyfikowanie podmiotów uczestniczących w procesach wspieranych przez system.

Uczestników dzielimy na dwie grupy: aktorów oraz pozostałych uczestników. Aktorem jest podmiot, który wchodzi w interakcję z systemem. Dobrze jest zidentyfikować wyraźnie również pozostałych uczestników, którzy nie są aktorami, ale mimo to uczestniczą w procesach wspieranych przez projektowany system.

Dla każdego uczestnika powinniśmy zidentyfikować jego nazwę oraz przyporządkować do grupy aktorów systemu lub pozostałych. Przykładowa tabela pozwalająca na dokumentację tej listy uczestników podana jest na rysunku 3.1.

Aktor	Nazwa	uwagi
Tak		
Nie		

Rysunek 4.1 Tabela uczestników z podziałem na aktorów systemu i pozostałych uczestników

Krok drugi

Po zidentyfikowaniu aktorów należy przejść do identyfikacji celów, jakie każdy z aktorów może realizować za pomocą systemu. Lista taka zwana jest listą Aktor-Cel i stanowi punkt wyjścia do identyfikacji przypadków użycia.

Tabelę przygotowaną do dokumentowania listy Aktor-Cel przedstawiliśmy na rysunku 3.2. Zawiera ona listę celów, które Aktor może realizować, oraz krótki dwu- trzyzdaniowy opis realizacji celu. Opis realizacji celu stanowi punkt wyjścia do opisu przypadku użycia i może być uważany za szkic opisu przypadku użycia.

Aktor	Cel	Szkic przypadku użycia

Rysunek 4.2 Lista Aktor-Cel wraz ze szkicem przypadków użycia

Krok trzeci

Po sporządzeniu listy Aktor-Cel warto jest dokonać przeglądu celów oraz przeprowadzić ich klasyfikację. Klasyfikację taką przeprowadza się, kierując się na przykład istotnością realizacji danego celu ze względu na priorytety biznesowe Aktora. Można tutaj posłużyć się skalą trójstopniową określającą ważność celu według skali: wysoka, średnia, niska. Jest to najczęściej najwygodniejsza i najłatwiejsza klasyfikacja priorytetów realizacji celów aktora. Można jednak stosować również inne skale, na przykład skalę pięciostopniową lub siedmiostopniową. Jednak klasyfikacje na więcej niż trzy klasy równoważności są trudniejsze i najczęściej na tym etapie nie są celowe.

Dokonanie klasyfikacji celów ze względu na ich priorytety ułatwia planowanie dalszych prac nad wymaganiami oraz cyklów wytwórczych (jeśli zdecydujemy się na model przyrostowy wytwarzania oprogramowania).

Rysunek 3.3 przedstawia tabelę dokumentującą listę Aktor-Cel z uwzględnieniem priorytetów celów aktora.

Aktor	Cel	Szkic przypadku użycia	Waga

Rysunek 4.3 Lista Aktor- Cel z uwzględnieniem priorytetów

Krok czwarty

Po opracowaniu listy Aktor-Cel oraz przypisaniu priorytetów można przystąpić do opisu poszczególnych przypadków użycia. Można przy tym wykorzystać priorytety celów, wybierając do opracowania cele w kolejności przypisanych priorytetów.

Przedstawimy teraz szablon, którym wygodnie jest się posługiwać w czasie dokumentowania przypadków użycia.

4.4.6. Konwencje typograficzne i opis szablonu

podkreślony tekst odwołanie do innego przypadku użycia, gdy tekst jest wyrażeniem czasownikowym, w przeciwnym wypadku jest to odwołanie do formatu danych

Zakres [Przypadek użycia czego?]

Poziom [Cel użytkownika, Streszczenie lub Podfunkcja]

Uczestnicy i interesy

[Nazwa uczestnika] [Ta część z jego interesów, którą przypadek użycia chroni i próbuje spełnić]

Aktor główny [Kto używa?]

Warunek początkowy [Asercja spełniona przy wchodzeniu do scenariusza]

Wyzwalacz [Zdarzenie, które powoduje wejście do scenariusza]

Scenariusz

Aktor rzeczownik

System rzeczownik

Rozszerzenia

1a. Awaria: [Co może pójść nie tak i jak te awarie są obsługiwane?]

Reguły biznesowe

1. [wymagania dziedziny problemowej, pytania do eksperta merytorycznego]

Jako przykład przedstawimy opis przypadku użycia *Zgłoś chęć uczestnictwa w turnieju* dla pewnego systemu obsługującego rozgrywki indywidualne w turnieju golfowym. System wspierający organizację tych rozgrywek składa się z kilku modułów. Jednym z nich jest moduł nazwany Witryną.

Zgłoś chęć uczestnictwa w turnieju

Zakres Witryna

Poziom Cel użytkownika

Uczestnicy i interesy

Gość witryny chce zagrać w turnieju

Organizator turnieju chce sprawnie skompletować uczestników

Aktor główny Gość witryny

Warunek początkowy Gość witryny wybrał turniej indywidualny, którego termin rejestracji nie upłynął

Wyzwalacz Gość witryny postanawia zgłosić chęć uczestnictwa w turnieju

Scenariusz

1. Gość witryny podaje swoje dane zawodnika oraz opcjonalne uwagi.
2. Witryna stwierdza, że wprowadzone dane spełniają wymagania uczestnictwa.
3. Witryna przekazuje dane zgłoszenia do serwera.
4. Serwer dodaje zawodnika do listy zawodników spoza OZG.
5. Serwer zapamiętuje zgłoszenie oraz jego datę.

Rozszerzenia

2a. Wprowadzone dane nie spełniają wymagań uczestnictwa:

2a1. Witryna powiadamia o sytuacji, jeśli gość witryny chce przerwać to zakończ.

3a. Serwer jest niedostępny:

3a1. Witryna informuje o problemie, zakończ.

4a. Zawodnik o takim imieniu, nazwisku i dacie urodzenia już istnieje na liście zawodników spoza OZG:

4a1. Serwer uaktualnia jego dane kontaktowe.

5a. Zawodnik o takim imieniu, nazwisku i dacie urodzenia już zgłosił chęć uczestnictwa w tym turnieju:

5a1. Witryna informuje o zdarzeniu i podaje datę zgłoszenia, zakończ.

Reguły biznesowe

Brak

4.5. Dokument Specyfikacji Wymagań

Po opisaniu przypadków użycia pora na zebranie wymagań w jednym dokumencie. Dokument taki nazywamy Dokumentem Specyfikacji Wymagań. Zawiera on zwięzły i jednoznaczny opis wymagań dla projektowanego systemu. Należy zadbać o to, aby był on czytelny i przejrzysty. Poniżej proponujemy Czytelnikowi strukturę Dokumentu Specyfikacji Wymagań wraz ze wskazówkami, jakie elementy należy umieścić w każdym z jego punktów.

Struktura Dokumentu Specyfikacji Wymagań

Wprowadzenie

[Wprowadzenie do **Dokumentu Specyfikacji Wymagań (DSW)** przedstawia zarys całego dokumentu. Zawiera cel dokumentu, jego zakres, definicje, akronimy oraz skróty, dokumenty powiązane i opis organizacji dalszej części dokumentu.]

[Uwaga: **DSW** zawiera kompletne wymagania wobec systemu lub podsystemu. Niniejszy dokument jest typowym szablonem **DSW** dla określania wymagań w oparciu o **model przypadków użycia (**ang. ***use-case*)**.]

Cel

[Ta sekcja przedstawia cel **Dokumentu Specyfikacji Wymagań (DSW)**. **DSW** w pełni opisuje zewnętrzne zachowanie aplikacji lub jej podsystemu. Zawiera także ograniczenia systemowe (wymagania niefunkcjonalne), ograniczenia projektowe i inne czynniki, których uwzględnienie jest niezbędne do utworzenia prawidłowej i wyczerpującej specyfikacji wymagań dla oprogramowania.]

Zakres

[Krótki opis składowych procesu wytworzenia oprogramowania, do których odwołuje się niniejszy **Dokument Specyfikacji Wymagań**.]

Definicje, akronimy i skróty

[Ta sekcja przedstawia wszystkie terminy, akronimy oraz skróty niezbędne do poprawnego zinterpretowania **Dokumentu Specyfikacji Wymagań.** Można tutaj zawrzeć odwołania do **Słownika**.]

Dokumenty powiązane

[Tutaj należy umieścić kompletną listę dokumentów, do których występują odwołania gdziekolwiek w obrębie **Dokumentu Specyfikacji Wymagań**. Dla każdego dokumentu podaj: tytuł, datę publikacji oraz organizację publikującą. Przedstaw także źródła, z jakich można go uzyskać.]

Organizacja dokumentu

[W tym miejscu należy umieścić informację o tym, co zawiera pozostała część **Dokumentu Specyfikacji Wymagań** oraz krótkie wyjaśnienie tego, jak dalszy materiał jest zorganizowany.]

Opis ogólny

[Ta sekcja **Dokumentu Specyfikacji Wymagań** opisuje ogólne czynniki, które wpływają na tworzony produkt oraz jego wymagania. Nie specyfikujemy tu wymagań – zamiast tego dostarczamy ogólny zarys (tło) dla ich definicji, którą przedstawiamy w sekcji 3. Dzięki informacjom zawartym w tej sekcji zrozumienie specyfikacji wymagań będzie łatwiejsze. Oto, co powinno się tutaj znaleźć: opis otoczenia, w jakim będzie działał produkt; co produkt będzie modelował z dziedziny problemowej; charakterystyki przyszłych użytkowników; ograniczenia (prawne, ekonomiczne etc.); założenia i zależności.]

Diagram przypadków użycia - wnioski

[W tym miejscu powinien znaleźć się ogólny opis diagramu przypadków użycia lub jego fragmentu odpowiedniego dla opisywanego podsystemu lub funkcji. W skład tego opisu musi wejść lista nazw i krótki ich opis dla wszystkich przypadków użycia oraz aktorów, z załączeniem odpowiednich diagramów i relacji pomiędzy nimi.]

Założenia i zależności

[Ta sekcja opisuje środki technologiczne mające wpływ na analizę funkcjonalną, dostępność gotowych podsystemów lub komponentów i inne założenia, które są istotne z punktu widzenia budowy **Dokumentu Specyfikacji Wymagań**.]

Wymagania systemowe

[Ta sekcja **Dokumentu Specyfikacji Wymagań** zawiera wszystkie wymagania systemowe na takim poziomie szczegółowości, aby projektanci mogli zbudować rozwiązanie zgodne z tymi wymaganiami, a testerzy mogli stworzyć plan testów sprawdzających realizację tych wymagań.]

Wymagania funkcjonalne

[Diagram przypadków użycia zawiera większość wymagań funkcjonalnych dotyczących systemu wraz z niektórymi wymaganiami niefunkcjonalnymi. Dla każdego zidentyfikowanego przypadku użycia należy załączyć szczegółowy opis w tej sekcji. Trzeba też upewnić się, że każde wymaganie jest jasno sprecyzowane.]

Wymagania niefunkcjonalne

[Wymagania niefunkcjonalne powinny zawierać te ograniczenia systemowe, które nie są bezpośrednio związane z funkcjonalnością produktu. Każde wymaganie niefunkcjonalne powinno być w prosty i czytelny sposób opisane. Ważne jest, żeby poziom szczegółowości opisu pozwalał na zweryfikowanie, czy finalny produkt spełnia postawione wymagania. W tym celu dobrze jest, żeby przy opisie ograniczenia podać miarę, pozwalającą na przetestowanie systemu pod jego kątem.]

Informacje pomocnicze

Informacje pomocnicze mają na celu ułatwienie korzystania z Dokumentu Specyfikacji Wymagań. Mogą zawierać:

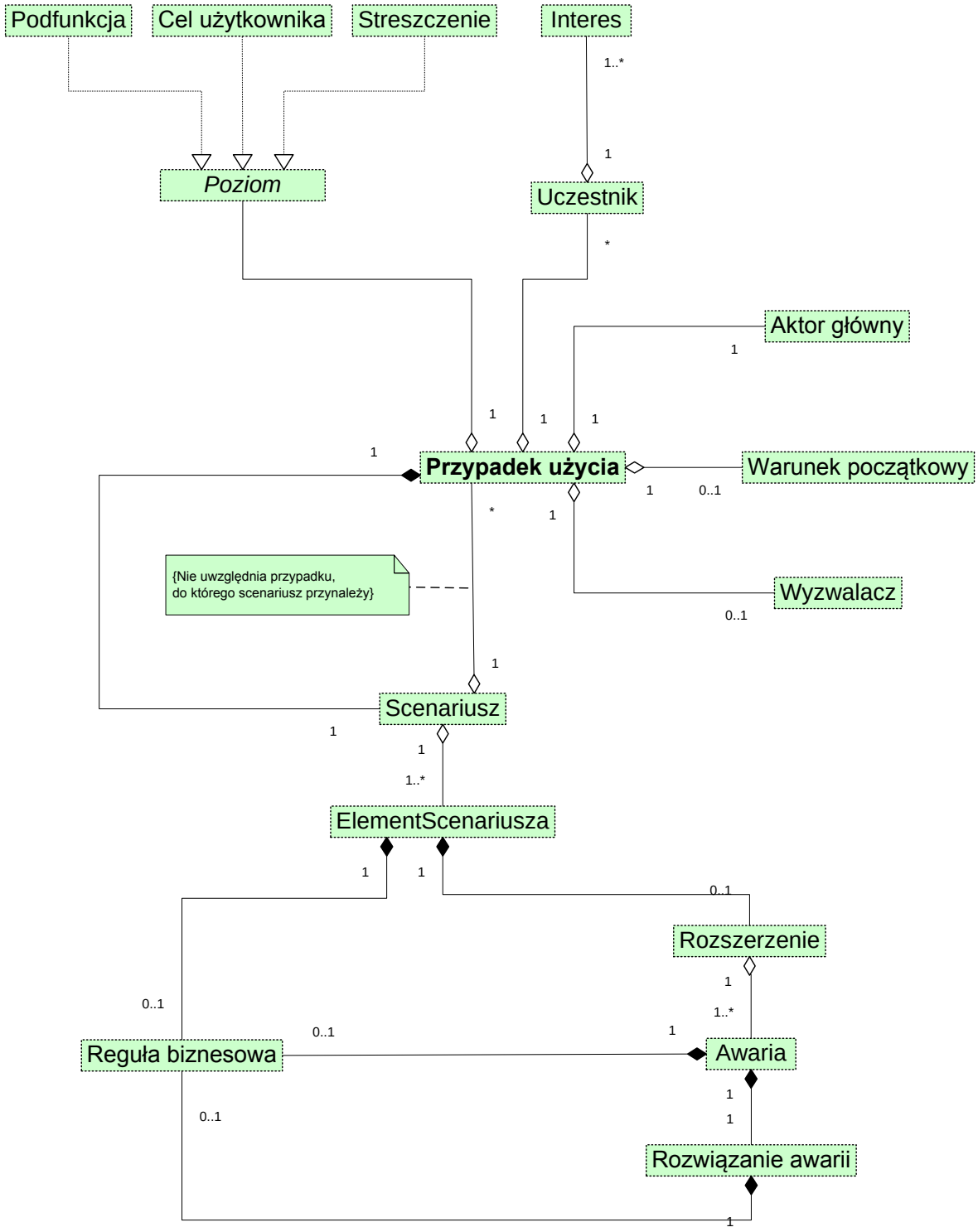
- spis treści,
- indeks,
- aneksy.

W szczególności można tutaj zawrzeć scenariusze dla przypadków użycia lub prototyp GUI. Gdy dołączymy w tym miejscu aneksy, należy jasno czy są one częścią Dokumentu Specyfikacji Wymagań, czy też nie.

4.5.1. Wzorzec szablonu specyfikacji wymagań funkcjonalnych

Poparzmy teraz na wzorzec szablonu opisu specyfikacji wymagań funkcjonalnych z punktu widzenia inżynieria wymagań. W tym celu na rysunku 4.1 przedstawiliśmy schemat specyfikacji wymagań

funkcjonalnych wyrażony w języku UML. Model ten może być pomocny podczas porządkowania struktury opisu wymagań oraz podczas tworzenia narzędzi wspierających prace analityka.



Rysunek 4.4 Schemat specyfikacji wymagań funkcjonalnych

Rysunek 4.4 przedstawia model logiczny szablonu scenariusza w specyfikacji wymagań funkcjonalnych. Model traktuje poszczególne elementy szablonu opisującego przypadek użycia jako obiekty. Obiektem wyjściowym jest obiekt przypadku użycia. Na przypadek użycia składają się następujące elementy:

- **Poziom** – określa, czy przypadek użycia jest celem użytkownika, streszczeniem, czyli ciągiem czynności, czy podfunkcją, czyli opisem algorytmu lub wewnętrznej funkcjonalności;
- **Uczestnicy i interesy** – określa uczestników oraz część interesów, które przypadek chroni i próbuje spełnić;
- **Aktor główny** – osoba lub komponent systemu realizujący przypadek użycia;
- **Warunek początkowy** – nie jest wymagany, określa asercję spełnioną przy wchodzeniu do scenariusza;
- **Wyzwalacz** – nie jest wymagany, określa punkt startowy, zdarzenie, które powoduje wejście do scenariusza;
- **Scenariusz** – określa ciąg czynności wykonywanych przez aktora lub system; scenariusz może również zawierać inne przypadki użycia;
- **Rozszerzenia** – określają sytuacje wyjątkowe, awarie oraz „lekarstwa" na niespodziewane zdarzenia;
- **Reguły biznesowe** – opisują wymagania dziedziny problemowej, zagadnienia merytoryczne.

4.6. Najczęstsze błędy popełniane przy modelowaniu wymagań

Na co należy zwrócić szczególną uwagę podczas modelowania wymagań? Jakie są najczęstsze błędy popełnianie na tym etapie?

Do najczęstszych błędów popełnianych podczas modelowania wymagań należą:

- nieprawidłowa identyfikacja celów aktora;
- brak identyfikacji uczestników niebędących aktorami;
- zbytnie rozdrobnienie celów;
- brak uwzględnienia scenariuszy pobocznych;
- brak jasnej struktury dokumentu wymagań;
- dokumentowanie wymagań niezrozumiałym i zagmatwanym językiem.

4.7. Ćwiczenia sprawdzające

4.7.1. Pytania testowe

1. Zbieranie i analiza wymagań (ang. *requirements capture and analysis*) jest fazą procesu wytwarzania oprogramowania:

a) która w praktyce nigdy nie obejmuje przedefiniowania procesów biznesowych;

b) której podstawowym artefaktem jest – z założenia – specyfikacja systemu dostosowana do technologii wybranej przez klienta;

c) podczas której analityk skupia się wyłącznie na rozpoznaniu dziedziny problemowej i oczekiwaniach klienta, pozostawiając kwestie ograniczeń technologicznych do fazy projektowania.

2. O związkach między przypadkami użycia możemy powiedzieć, że:

a) dla zachowania elastyczności modelu UC przypadek bazowy – z założenia – nie powinien nic „wiedzieć" o przypadkach rozszerzających («extend»);

b) po wykonaniu załączanego («include») UC możemy zasadniczo wrócić do dowolnego miejsca scenariusza bazowego UC;

c) związek «include» pozwala wyodrębnić elementy ponownego użycia.

3. W trakcie zbierania wymagań należy identyfikować:

a) tylko wymagania głównego aktora;

b) tylko ograniczenia;

c) cele aktorów i ograniczenia.

4.7.2. Pytania otwarte

1. Zastanów się, w jaki sposób najlepiej jest pozyskiwać wymagania od przyszłych użytkowników systemu. Wymień i scharakteryzuj kilka metod.

2. Czym różnią się wymagania funkcjonalne od wymagań niefunkcjonalnych?

3. Czy znasz inne metody dokumentowania wymagań, które nie korzystają z metody przypadków użycia? Wymień je.

4.8. Słowniczek pojęć wprowadzonych w rozdziale

Aktor – podmiot (np. użytkownik lub inny system), który wchodzi w interakcję z systemem.

Uczestnik – podmiot biorący udział w procesach biznesowych wspieranych przez system. Uczestnik może być aktorem systemu lub nie (jeśli nie wchodzi w interakcję z systemem).

Przypadek użycia – specyfikacja gwarantująca aktorowi wykonanie przez system usługi realizującej cel aktora.

4.9. Przegląd literatury

3.1. D. Leffingwell, D. Widrig, *Zarządzanie wymaganiami*, WNT Warszawa, 2003

W książce tej Czytelnik znajdzie obszerne omówienie problematyki inżynierii wymagań. Nie znajdziecie w niej jednak diagramów UML, ale opis różnych aspektów zarządzania wymaganiami, zarządzania zmianą, cykle życiowym itd. Pozycję tę polecamy wszystkim, których szczególnie zainteresowała problematyka zarządzania wymaganiami.

3.2. A. Cockburn, *Jak pisać efektywne przypadki użycia*, WNT, Warszawa, 2004

Jest to jedna z najlepszych pozycji traktująca o pozyskiwaniu i dokumentowaniu wymagań z wykorzystaniem przypadków użycia, a jednocześnie nasza ulubiona pozycja z tego zakresu. Polecamy ją każdemu, kto chciałby dokumentować wymagania z wykorzystaniem przypadków użycia.